INTELLIGENT SOFTWARE ENGINEERING

ISE-A3

Repository Investigation and Reproducible Build

Project: Two-Stage Clinic Appointment and Arrival-Confirmation System

A continuation of the same project used in ISE-A1 and ISE-A2.

Course	Intelligent Software Engineering
Assignment	ISE-A3 — Repository Investigation and Reproducible Build
Course release	19 October 2026, 16:30 (Asia/Shanghai)
Deadline	26 October 2026, 16:30 (Asia/Shanghai)
Maximum marks	5
Investigated snapshot	Git tag: ise-a2-v1.0
Commit	e8612772f9a145de60bbaf288b43950ae64d59d6
Execution environment	Python 3.13.5 · Linux x86_64 · glibc 2.41
Investigation date	25 October 2026

Investigation result at a glance

The frozen A2 reference model was set up in a fresh Python virtual environment. Syntax compilation succeeded, all 36 unit tests passed, and the seven-cell model walkthrough notebook executed successfully with six of six boundary examples passing. The repository is a deterministic reference model, not a deployable clinic service; this boundary is central to the risk and ADR findings.

Prepared as a repository investigation report with execution evidence, architectural observations, a risk register and an Architecture Decision Record.

1. Executive Summary

This investigation examines the ISE-A2 repository snapshot tagged ise-a2-v1.0 at commit e8612772f9a145de60bbaf288b43950ae64d59d6, continuing the clinic appointment project from ISE-A1 and ISE-A2. The repository implements a deterministic Python reference model for appointment windows, check-in decisions, appointment-state coherence, queue-ticket rules, no-show authorization and ticket-state transitions. It does not implement a complete web application or a production clinic service.

A fresh virtual environment using Python 3.13.5 was created. Installing the repository's requirements caused no third-party runtime packages to be installed; the model and unit tests rely on the Python standard library. Syntax compilation completed successfully, and python -m unittest discover -s tests -v passed all 36 tests. The seven-cell A2 model walkthrough notebook was also executed successfully. Its six deterministic boundary examples all matched the expected check-in decisions.

The main architecture finding is that the domain rules are implemented as pure functions and data classes. This makes rule behavior easy to test, but the repository has no API, persistence adapter, application service, or production start command. Consequently, the check-in-to-ticket path can be reconstructed only partly from implemented code; state mutation and durable ticket creation are integration responsibilities not present in this snapshot. The highest risks are duplicate tickets under concurrent requests, missing end-to-end orchestration, and the absence of production identity/access-control and persistence boundaries. The ADR recommends enforcing ticket uniqueness atomically at the persistence boundary, retaining the pure model as a testable rule layer. The successful tests demonstrate conformance to implemented examples, not clinical effectiveness or production readiness.

Scope of the evidence

All execution claims in this report are based on commands run against the repository snapshot identified above. Test fixtures are synthetic and deterministic. No real patient data, target-clinic workflow, external service, database, UI, or clinical deployment was exercised.

2. Repository Identity and Frozen Scope

2.1 Snapshot identity

Item	Observed value	Evidence / interpretation
Branch	master	git status --short --branch showed branch master with no working-tree changes.
Tag	ise-a2-v1.0	Annotated in the repository as the A2 domain model and testable specification version.
Commit	e8612772f9a145de60bbaf288b43950ae64d59d6	HEAD at inspection; commit subject: “Clarify A2 manifest metadata”.
Previous commits	0447a07; 28ef948; f2cf883; 690d124	Manifest update, A2 final specification, A2 working draft and A1 artifact respectively.
Project continuity	ISE-A1 → ISE-A2 → ISE-A3	Same proposed two-stage clinic appointment and arrival-confirmation workflow.

2.2 Repository inventory

Area	Key files	Role in the architecture
Domain implementation	src/clinic_model.py	Enums, data classes, time-window checks, validation and decision functions.
Automated tests	tests/test_clinic_model.py	36 unittest cases covering boundaries, state coherence, queue-ticket rules and authorization.
Executable walkthrough	notebook/a2_model_walkthrough.ipynb	Runs six check-in boundary examples and invokes the test suite.
Domain documentation	models/*.mmd, *.dot, *.csv	Context, event flow, state model, data dictionary, invariant list and check-in decision table.
Evidence/results	raw_results/*, results/a2_artifact_manifest.json	Environment record, prior test/notebook logs and artifact manifest.
Project guidance	README.md, src/README.md, docs/*	Reproduction command, intended scope, limits, walkthrough notes and AI-use disclosure.
Runtime/dependency config	requirements.txt	States Python 3.11+ and no third-party requirements for model/tests; it is not a full lockfile.

2.3 Repository boundary

The inspected snapshot is a rule model and supporting evidence package. It is not an integrated patient-facing product. The only application logic observed is in the Python model module; the automated test file is executable as a test runner, and the notebook is an analysis entry point. No web framework, HTTP route, database driver, ORM, persistence adapter, frontend, deployment descriptor, or production service entry point was found in the tracked files.

3. Reproducible Setup, Build, Test and Start Commands

3.1 Frozen runtime/environment

Property	Observed / configured value	Assessment
Python runtime	Python 3.13.5 (CPython)	Exact runtime used for the clean virtual-environment test.
OS/platform	Linux x86_64; glibc 2.41	Observed execution environment; platform kernel in the run environment reported Linux 6.18.55.
External runtime libraries	None for model/tests	Imports are from the Python standard library: dataclasses, datetime, enum and collections; tests use unittest.
Requirements file	Python 3.11 or newer; no package entries	Sufficient for the model's standard-library dependencies, but it does not pin patch version or optional notebook tooling.
Input data	Synthetic deterministic fixtures	No random seed is needed; no random generator or real patient data is used.
Notebook tooling	Jupyter/nbconvert present in notebook-execution environment	Optional notebook tooling is not specified as a pinned dependency in requirements.txt.

3.2 Clean setup and syntax/build check

The following commands were executed from the repository root. A new virtual environment was created with Python 3.13.5; the requirements installation completed without installing third-party runtime dependencies.

```
python -m venv .venv
# Linux/macOS
. .venv/bin/activate
python --version
python -m pip --version
python -m pip install -r requirements.txt
python -m compileall -q src tests
```

Command	Observed result	Status
python --version inside clean venv	Python 3.13.5	PASS
python -m pip install -r requirements.txt	Completed; requirements file specifies no third-party model/test packages.	PASS
python -m compileall -q src tests	Exit code 0; no syntax errors printed.	PASS

3.3 Test command and actual result

```
python -m unittest discover -s tests -v
```

Result field	Actual captured result
Test cases executed	36
Failures / errors	0 / 0
Runtime	0.002 seconds in the recorded fresh-venv run
Terminal summary	Ran 36 tests in 0.002s | OK

Interpretation

The passing result establishes that the current test cases agree with the current Python reference rules. It does not demonstrate performance under concurrent use, correctness of a database transaction, security authorization in a real identity system, or improvement in patient waiting times.

3.4 Notebook reproduction

```
jupyter nbconvert --to notebook --execute notebook/a2_model_walkthrough.ipynb \
  --output a2_model_walkthrough_executed.ipynb \
  --output-dir /tmp --ExecutePreprocessor.timeout=120
```

The command was run from the repository root. The executed notebook contains seven cells, completed without cell errors, and produced the expected walkthrough outputs. All six boundary cases passed: one minute before the window, exactly at the start, inside the window, exactly at the end, inside the grace interval, and at the grace threshold. The notebook writes its JSON result and unittest transcript into raw_results/ when run from a writable checkout.

3.5 Build and start-command assessment

Lifecycle step	Repository finding	Required interpretation
Build	No packaging/build configuration such as pyproject.toml, setup.py or compiled application target is present. compileall is used as a syntax/build sanity check.	No separate build artifact is produced.
Start	No production main, API route, WSGI/ASGI app, CLI workflow or database-backed service is defined.	There is no valid clinic-service start command for this snapshot; the test runner and notebook are the executable entry points.
Clean setup	Fresh virtual environment and requirements-file processing were completed.	The runtime test path has no external model/test dependency.
Test	36 unit tests passed in the fresh virtual environment.	Keep the command and output with the versioned evidence.

3.6 Warnings and negative-result handling

One notebook-execution attempt supplied a configuration flag that the installed nbconvert version did not recognise. The command emitted a warning but still completed. To remove ambiguity, the notebook was rerun from the repository root without that unsupported option; the clean rerun wrote the executed notebook and showed no cell errors. This warning is retained here rather than being represented as a failed test. No failing unit tests occurred in the fresh-venv run. Negative cases are intentionally covered by tests for invalid/reversed time windows, naive timestamps, duplicate active tickets, unauthorised no-show recording, and invalid ticket transitions.

4. Architecture and Call-Path Map

The repository follows a small, test-first reference-model structure. The arrows below show observed source relationships and execution flow, not production network calls.

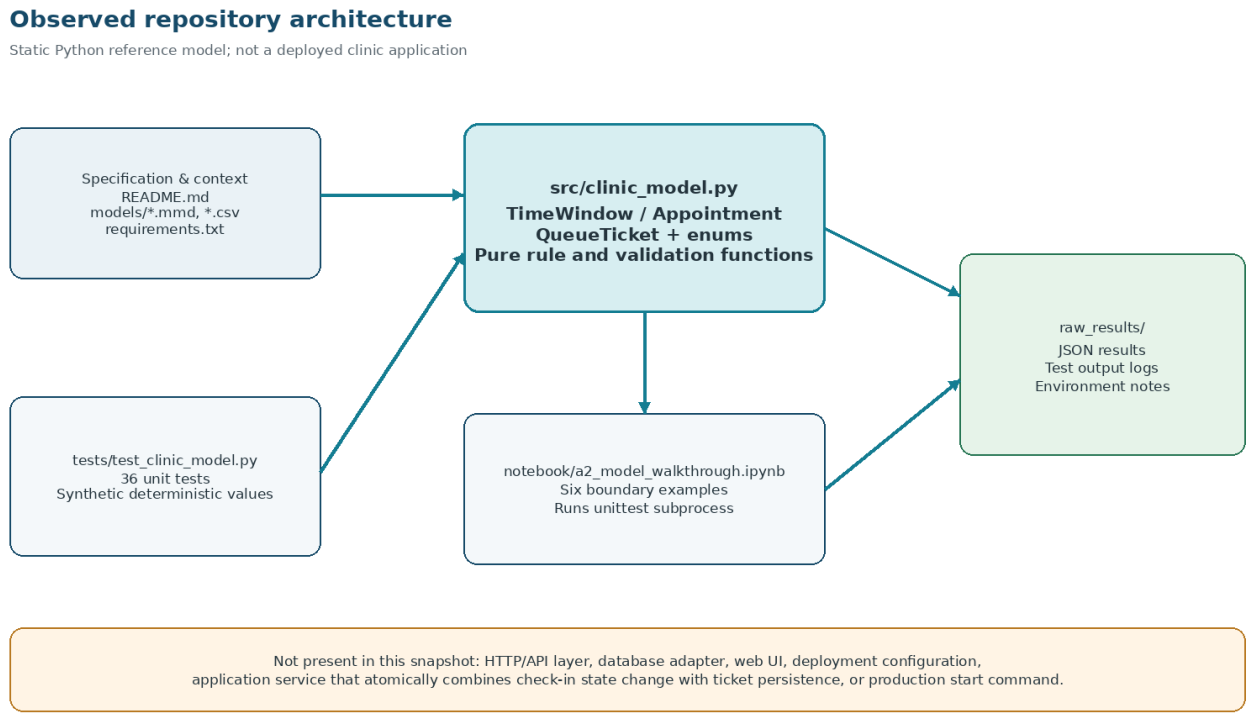


Figure 1. Observed module and evidence layout. The bottom boundary is an explicit finding: the repository has no persistence or live application layer.

4.1 Entry points, modules and dependencies

Component	Observed responsibility	Dependencies / callers
tests/test_clinic_model.py	Unittest entry point; 36 assertions/tests over the model.	Imports model symbols from src.clinic_model; Python unittest and datetime.
src/clinic_model.py	Core rule layer. Defines AppointmentState, TicketState, CheckInDecision, TimeWindow, Appointment, QueueTicket and validation/decision functions.	Python standard library only; no imports to web/database services.
notebook/a2_model_walkthrough.ipynb	Deterministic boundary scenario; invokes unittest as a subprocess; saves JSON and text outputs.	Requires notebook execution tooling beyond the core runtime; subprocess calls the test command.
models/ and docs/	Static diagrams, decision table, dictionary, invariants and scope/limitations.	Documentation inputs; not imported by runtime source.
raw_results/	Stores environment and execution records.	Written by the notebook/run process; not a persistence layer for application data.

4.2 Critical domain functions

Function / type	Observed contract	Safety-relevant detail
TimeWindow.is_valid() / .contains()	Requires timezone-aware start/end and positive duration; models [start, end).	Start boundary is accepted; end boundary is excluded.
decide_check_in(window, state, now, grace)	Returns a CheckInDecision; validates time window, timezone awareness and non-negative grace.	Pure decision: it does not mutate an appointment or issue a ticket.
appointment_violations(a, grace)	Reports inconsistent identifiers, timestamps and state/time combinations.	CHECKED_IN/COMPLETED require a timestamp; NO_SHOW must be after window end plus grace.
can_issue_ticket(a, existing)	Allows issuance only for a coherent CHECKED_IN appointment and no active ticket for that appointment.	The check inspects the provided in-memory ticket list; it is not an atomic persistence guarantee.
ticket_registry_violations(tickets)	Checks ticket id uniqueness, non-empty ids, active-ticket uniqueness, session/queue number uniqueness and aware issue time.	Validates a collection in memory; no durable store is present.
can_record_no_show(a, now, actor_role, grace)	Requires confirmed, not checked in, past grace, valid appointment and allowed role name.	actor_role is a string value; external identity/authentication is not implemented.
ticket_transition_allowed(current, target)	Checks the declared finite set of permitted ticket-state transitions.	No event bus, persistence transaction or queue display consumes the transition.

4.3 Configuration, persistence and external interfaces

- Configuration: GRACE_MINUTES_DEFAULT = 15 is hard-coded as an explicit prototype assumption. The README labels it a clinic-configurable value, not a verified clinic policy.
- Persistence: no database module, schema/migration, repository adapter or transaction boundary is present.
- External interfaces: no HTTP client/server, messaging service, kiosk integration, doctor display, SMS/WeChat notification, or external identity provider is present in the tracked source.
- Security boundary: the model uses an allowed-role string set for no-show eligibility; this is a domain rule, not proof of authentication or authorization in a deployed system.
- Data: test records use synthetic IDs and timestamps; the repository does not include real patient records.

5. Reconstructed End-to-End Path

The path below reconstructs what the current model can support for a patient who booked 09:00–11:00 and requests check-in at 10:15 (+08:00). “Fact” means directly evidenced by code/tests. “Inference” means a higher-level integration step implied by the intended workflow but not implemented in this repository snapshot.

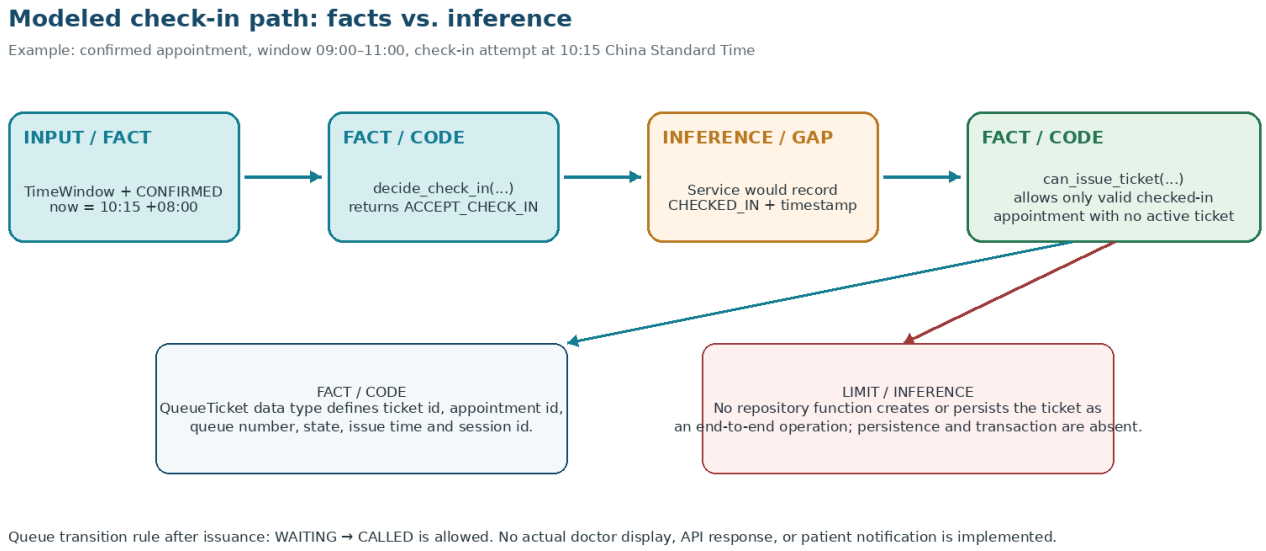


Figure 2. A modeled path from check-in decision to queue-ticket eligibility, with the missing orchestration boundary identified.

Step	Evidence type	Reconstruction and result
1. Existing confirmed booking	Assumption / test fixture	An Appointment has state CONFIRMED and a valid timezone-aware TimeWindow from 09:00 to 11:00.
2. Patient attempts check-in at 10:15	Input / synthetic scenario	decide_check_in(window, CONFIRMED, now) is called with a timestamp inside the half-open window.
3. Check-in decision	Fact: code + test	The decision function returns ACCEPT_CHECK_IN; the notebook shows the 10:15 case passed.
4. Record checked-in state and timestamp	Inference / absent orchestration	An application service would need to save a new Appointment with state CHECKED_IN and an aware checked_in_at. The repository does not implement this transition as a mutating command.
5. Evaluate ticket eligibility	Fact: code	can_issue_ticket(appointment, existing_tickets) returns true only if the appointment is coherently checked in and no active ticket exists for it.
6. Construct and assign a ticket	Partial fact / integration gap	The QueueTicket type contains ticket id, appointment id, queue number, state, issue time and session id. There is no repository function that atomically creates and persists it.
7. Move ticket to called	Fact: rule; no external effect	ticket_transition_allowed(WAITING, CALLED) returns true. A display or doctor-call action is not implemented.

5.1 Facts separated from inference

Facts directly observed

The model is deterministic; decide_check_in is pure; the start is inclusive and the end exclusive; grace-threshold eligibility does not automatically record NO_SHOW; ticket eligibility requires a valid checked-in appointment; one active ticket per appointment is validated for the supplied ticket collection; 36 unit tests passed.

Inferences / not evidenced as implemented

A user interface, booking API, service operation that persists the state transition, durable queue-ticket creation, transaction/locking semantics, staff login, audit-event persistence, doctor display and patient notification are not present. Those are requirements for a future integration layer, not current runtime capabilities.

6. Risk Register

Risks are ranked for the repository as inspected, not for a hypothetical production system that has not been implemented. Severity reflects likely impact if the reference model were integrated without addressing the risk.

ID / risk	Evidence and trigger	Likelihood / impact	Mitigation / safe change boundary
R1 — Duplicate queue tickets under concurrent/retried requests	can_issue_ticket checks a supplied in-memory list. Two concurrent requests could both pass a pre-check before either persists a ticket. There is no database transaction or uniqueness constraint in the repository.	Likelihood: Medium after integration. Impact: High.	Retain the pure guard but enforce atomic issuance in a persistence transaction. Add durable uniqueness constraints for active ticket per appointment and queue number per session, plus an idempotency key. Test concurrent/repeated requests.
R2 — No integrated end-to-end state transition	decide_check_in returns a decision only; there is no service/command that atomically records CHECKED_IN and creates a QueueTicket. No server start command exists.	Likelihood: High if reused as a complete app. Impact: High.	Keep this repository as the rule layer. Add an explicit application-service boundary that validates, records state and issues a ticket transactionally. Add service-level tests before introducing a UI.
R3 — Authentication/privacy not integrated	can_record_no_show receives an actor_role string. No authentication provider, session management, audit persistence or patient-data access layer exists.	Likelihood: Medium during integration. Impact: High.	Do not expose model functions directly to users. Derive role from an authenticated principal; enforce least privilege; audit sensitive actions; minimise identifiers; define retention/deletion with the clinic and applicable rules.
R4 — Runtime/tooling not fully locked	requirements.txt says Python 3.11+ but does not pin the exact Python patch version or optional Jupyter/nbconvert packages. Fresh-venv unit tests work because core code uses only the standard library.	Likelihood: Medium across machines. Impact: Medium.	Record the tested Python version and platform, pin notebook tooling separately if notebooks are part of the assessed reproducible path, and use a CI matrix for the supported Python range.

6.1 Risk priority

R1 and R2 should be addressed before treating the workflow as a usable service. R3 must be resolved before any real patient data is introduced. R4 is lower in immediate runtime risk because the reference model has no third-party dependencies, but it matters for repeatable notebook execution and cross-machine reproduction.

7. Architecture Decision Record (ADR-001)

ADR-001: Enforce queue-ticket uniqueness at an atomic persistence boundary

Field	Decision record
Status	Accepted as the target integration decision; persistence is not implemented in the investigated snapshot.
Context	The model checks whether a ticket may be issued by inspecting the current in-memory ticket list. This is useful for a deterministic model but is not sufficient to prevent race conditions once multiple kiosk or staff requests can run concurrently.
Decision driver	Prevent duplicate active queue tickets and duplicate queue numbers when repeated or concurrent check-in requests occur.

7.1 Alternatives compared

Alternative	Advantages	Disadvantages / risk	Assessment
A. Application-only pre-check	Minimal extra infrastructure; preserves simple pure functions; quick for a prototype.	Two requests can read the same pre-state and both pass before either writes; in-memory checks do not survive restarts; correctness depends on caller sequencing.	Not sufficient for multi-user production operation. Keep it as an early validation layer only.
B. Transactional persistence constraint + model guard	Atomic creation; durable uniqueness; can support idempotent retries; model validation remains easy to test.	Adds a storage schema, transaction logic and integration tests; chosen database must support the needed constraint/transaction semantics.	Chosen for the future service boundary because it directly addresses the high-impact race condition.

7.2 Consequences and implementation outline

- Keep can_issue_ticket and ticket_registry_violations as fast domain-level checks and specification aids.
- Introduce a service operation that starts a transaction, reloads/locks the booking as needed, validates checked-in state, claims the ticket idempotently, and inserts ticket + audit event atomically.
- Enforce durable invariants in storage: unique ticket identifier; unique queue number within a queue session; and at most one active ticket per appointment, using a supported conditional/partial unique constraint or an equivalent transactional design.
- On a retry, return the existing ticket for the same idempotency key instead of creating a second ticket.
- Add service-level integration tests for simultaneous requests, duplicate retries, cancellation/late-arrival conflict, transaction rollback and storage failures.
- Do not add a database to this reference-model repository merely to make it appear more complete; introduce persistence only at the architecture stage where its behaviour can be tested and reviewed.

Why this is the safer change boundary

The current source provides a pure decision layer that is cheap to test. A transactional storage boundary should be introduced as a separate layer, so storage-specific locking and uniqueness do not contaminate the deterministic domain rules. The decision is based on repository structure and a concurrency failure mode; it has not been benchmarked against a real clinic workload.

8. Execution Evidence Ledger

Evidence ID	Artifact / command	Observed result	Classification
E1	git status --short --branch and git log --oneline --decorate -8	Branch master; clean status; HEAD/tag ise-a2-v1.0; commit SHA recorded on cover.	Direct observation
E2	python --version; python -m pip --version; platform introspection	Python 3.13.5; pip 25.1.1 in fresh virtual environment; Linux x86_64 / glibc 2.41.	Direct observation
E3	python -m pip install -r requirements.txt	Completed with no third-party runtime packages added for model/tests.	Direct observation
E4	python -m compileall -q src tests	Completed successfully without syntax errors.	Direct observation
E5	python -m unittest discover -s tests -v	36 tests; 0 failures and 0 errors; final summary OK.	Direct observation
E6	Executed notebook/a2_model_walkthrough.ipynb	Seven cells; no cell errors; all six boundary cases matched expected decisions; notebook wrote results JSON and test transcript.	Direct observation
E7	Search for entry points and external integrations across tracked files	No API/web-server/database adapter or production start entry point found.	Direct observation bounded by tracked snapshot
E8	Check-in → state transition → ticket sequence	First and ticket-eligibility rules exist; the orchestration and durable write are inferred missing integration responsibilities.	Mixed: fact + explicitly labelled inference

8.1 Reproduction transcript

```
Python 3.13.5
$ python -m compileall -q src tests
# exit code: 0 (no output)
$ python -m unittest discover -s tests -v
... [36 individual tests reported ok] ...
----------------------------------------------------------------------
Ran 36 tests in 0.002s

OK
```

8.2 Evidence retention

The report records the exact command and summary rather than replacing the source with a screenshot. The repository already contains earlier raw test/notebook output files. The fresh run described here is recorded separately in this report. No negative unit-test run occurred in this clean execution; invalid inputs and forbidden transitions remain represented as negative test cases. The initial notebook invocation warning and the clean rerun are disclosed in Section 3.6.

9. Safe Change Boundary and Next Engineering Gate

9.1 Safe changes within the current repository

- Improve pure domain validation or clarify decision outcomes while preserving the documented half-open window semantics and no-show grace rule.

- Add tests for missing cases or property-based checks without introducing real patient data.
- Improve README/reproduction instructions and pin optional notebook tooling if required for the assessed path.
- Keep application/service and persistence concerns separate from the domain model.

9.2 Changes that require extra review

- Changing the meaning of arrival-window endpoints or the grace threshold, because this changes appointment policy and test expectations.
- Adding patient identifiers, medical information, staff accounts, database persistence or external messaging integrations.
- Implementing concurrency-sensitive ticket issuance without transactional constraints and failure-mode tests.
- Adding a real clinic workflow, production deployment or legal-compliance claim without clinic approval and a documented review.

9.3 Acceptance criteria before this becomes an end-to-end prototype

Gate	Acceptance evidence
Service boundary	A single operation coordinates accepted check-in, persisted state and ticket issue with clear failure semantics.
Persistence	Schema/migrations and transaction design enforce uniqueness and support idempotent retry.
Integration tests	Concurrent duplicate requests, retries, cancellation conflict and rollback are tested against the chosen storage.
Authorization/privacy	Authenticated staff roles, least-privilege access, audit logging and retention/deletion rules are reviewed.
Operational fallback	Kiosk/network failure has a documented manual process and safe reconciliation path.
Reproducible release	Pinned runtime/dependencies, CI results, stable commit/tag and independent reproduction instructions exist.

10. AI-Use and Verification Log

Item	Disclosure
AI tool	ChatGPT (OpenAI).
Tasks assisted	Explain the A3 brief; inspect repository inventory and Python module structure; identify call paths and absent integration layers; draft the risk register and ADR options; structure and edit this report.
Suggestions accepted	Use the existing tagged A2 snapshot; treat pure decision functions separately from state mutation; distinguish direct code facts from inferred service behaviour; use actual clean-environment command outputs; recommend transactional uniqueness for multi-request ticket issuance.
Suggestions rejected or constrained	No claim that this is a live clinic application; no invented GitHub URL; no invented user interview, clinic measurement, production deployment, database execution, concurrency benchmark or legal approval.
Independent verification performed	Inspected tracked source/documentation; checked Git HEAD/tag and worktree; created a fresh venv; processed requirements; ran compileall; executed 36 unit tests; executed the notebook and checked cell outputs and six boundary cases.

Item	Disclosure
Remaining verification limits	No real clinic representative reviewed the model; no patient data was used; no external API/database, concurrency/load/security test or production deployment was exercised.

11. Conclusion and Limitations

The repository is reproducible for its current purpose: a deterministic reference specification for the clinic appointment domain. The frozen snapshot, clean virtual environment, syntax check, unit tests and notebook walkthrough provide repeatable evidence for the implemented rules. The architecture is small and easy to inspect because the model has no external runtime dependencies and uses pure functions for the key decisions.

The central limitation is also clear: the repository is not an end-to-end clinic system. The model can decide whether a check-in should be accepted and can evaluate whether a ticket is eligible, but it does not persist the state transition, atomically issue a ticket, authenticate staff, communicate with a kiosk or doctor display, or start a service. Treating this model as production-ready would overstate the evidence. The safe next boundary is a separately tested application-service and transactional persistence layer, while keeping the core domain rules independently testable.

These findings support a reversible engineering approach: preserve the current tagged model, add integration behaviour behind explicit interfaces, test the high-risk concurrency cases, and only then reassess whether the full workflow is ready for a clinic trial.

Appendix A. Platform Result Summary (150–300 Words)

This repository investigation examines the clinic appointment project carried from ISE-A1 and ISE-A2. The frozen source snapshot is tag ise-a2-v1.0, commit e8612772f9a145de60bbaf288b43950ae64d59d6. It contains a deterministic Python reference model, 36 unit tests, a model-walkthrough notebook, domain diagrams, a data dictionary, invariants, a requirements file and prior execution records.

A fresh Python 3.13.5 virtual environment was created on Linux x86_64. Processing requirements.txt did not add third-party runtime packages for the model or tests. python -m compileall -q src tests completed successfully, and python -m unittest discover -s tests -v passed all 36 tests with no failures or errors. The seven-cell walkthrough notebook executed without cell errors; all six deterministic boundary cases matched their expected check-in decisions.

The source architecture is a pure domain-rule layer: time windows, appointment state checks, check-in decisions, no-show eligibility, ticket validation and queue-ticket transitions. No HTTP/API layer, database adapter, production service entry point or application orchestration was found. Therefore, the check-in-to-ticket path is only partly implemented: ticket eligibility is checked in code, but state mutation and durable ticket issuance belong to a missing integration boundary.

The primary risk is duplicate active queue tickets under concurrent or repeated requests. ADR-001 selects transactional persistence constraints and idempotent ticket issuance for the future service boundary, retaining model-level validation as defence in depth. The tests verify current rule examples, not clinic effectiveness, security compliance or production readiness. All test values are synthetic; no patient records or target-clinic operational data were used.

Word count: 240

Appendix B. Key Repository References

Reference	Use in this report
README.md	Project continuity, reproduction command, limitations, synthetic-data statement and source/AI transparency.
requirements.txt	Declared minimum Python version and absence of external model/test runtime dependencies.
src/clinic_model.py	Direct source for model types, time-window policy, decision functions, invariant checks and transitions.
tests/test_clinic_model.py	Executable test cases and examples of positive, boundary and violating conditions.
notebook/a2_model_walkthrough.ipynb	Six deterministic check-in boundary examples and subprocess test execution.
models/data_dictionary.csv; models/invariants.csv; models/check_in_decision_table.csv	A2 domain artefacts used to confirm expected concepts and rule semantics.
raw_results/environment.txt; raw_results/a2_test_run.txt; raw_results/a2_notebook_test_output.txt	Prior recorded environment and test/notebook results; compared with the fresh run in this investigation.
docs/AI_use_log_A2.md; docs/model_walkthrough_record.md	Existing disclosure and explicit distinction between desk-based walkthrough and external validation.

Final evidence statement

The report records actual local repository inspection and execution evidence. It does not assert a live clinic trial, public service deployment, database integration, production security review or measured patient/clinic outcome.